

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 05/Unit 05: RAII, Rule of 3, and Destructors

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk

We will resume building a network of friends in this class.



Static Class Data

- If a data item in a class is declared as static, only one such item is created for the entire class, no matter how many objects there are.
- A static data item is useful when all objects of the same class must share a common item of information.
- A member variable defined as static has characteristics similar to a normal static variable:
- It is visible only within the class, but its lifetime is the entire program. It continues to exist even if there are no objects of the class.



RAII



RAII

- Stands for Resource Allocation Is Initialization
- It's a design principle that dictates
 - Manage resources by tying them to object's lifetime
- It means you should constructor and destructor to acquire and release resources automatically

Rule of 3



Rule of 3

- This is another design rule that states that if a class defines one (or two) of the following, then it should explicitly define all three
 - Destructor
 - Copy Constructor
 - Copy Assignment Operator

Rule of 3

The problem with default constructors

C++ has its own default copy constructor and copy operator, but they are not reliable



The Copy Constructor

- Let's mention another way to initialize an object: you can initialize it with another object of the same type.
- Surprisingly, you don't need to create a special constructor for this; one is already built into all classes.
- It's called the **default copy constructor**.

```
int main()
{
    Distance dist1(11, 6.25);           //two-arg constructor
    Distance dist2(dist1);              //one-arg constructor
    Distance dist3 = dist1;             //also one-arg constructor
}
```



The Copy Constructor

- This causes the default copy constructor for the Distance class to perform a member-by-member copy of `dist1` into `dist2`.
- A different format has exactly the same effect, causing `dist1` to be copied member-by-member into `dist3`

```
Distance dist3 = dist1;
```



Copy Constructor: Shallow Vs Deep Copy

- In C++, a copy constructor typically makes a **shallow copy** by default.
- This means that it copies the values of member variables directly.
- If a class contains pointers or dynamically allocated memory, the default copy constructor will copy the pointer values.
- This leads to both the original and the copied object to point to the same memory.
- This can cause issues like double deletion or unintended side effects.



Copy Constructor: Shallow vs Deep Co

- For a **deep copy**, (where the copy constructor allocates new memory and copies the contents of the original object's pointed-to data) we need to define a copy constructor.
- Observe the syntax.

```
class MyClass {  
public:  
    int* data;  
  
    // Default constructor  
    MyClass(int value) {  
        data = new int(value);  
    }  
  
    // Custom copy constructor for deep copy  
    MyClass(const MyClass& other) {  
        data = new int(*other.data);  
        // Allocate new memory and copy the value  
    }  
  
    // Destructor  
    ~MyClass() {  
        delete data; // Clean up the allocated memory  
    }  
};
```

Rule of 3

Custom “=” Operator

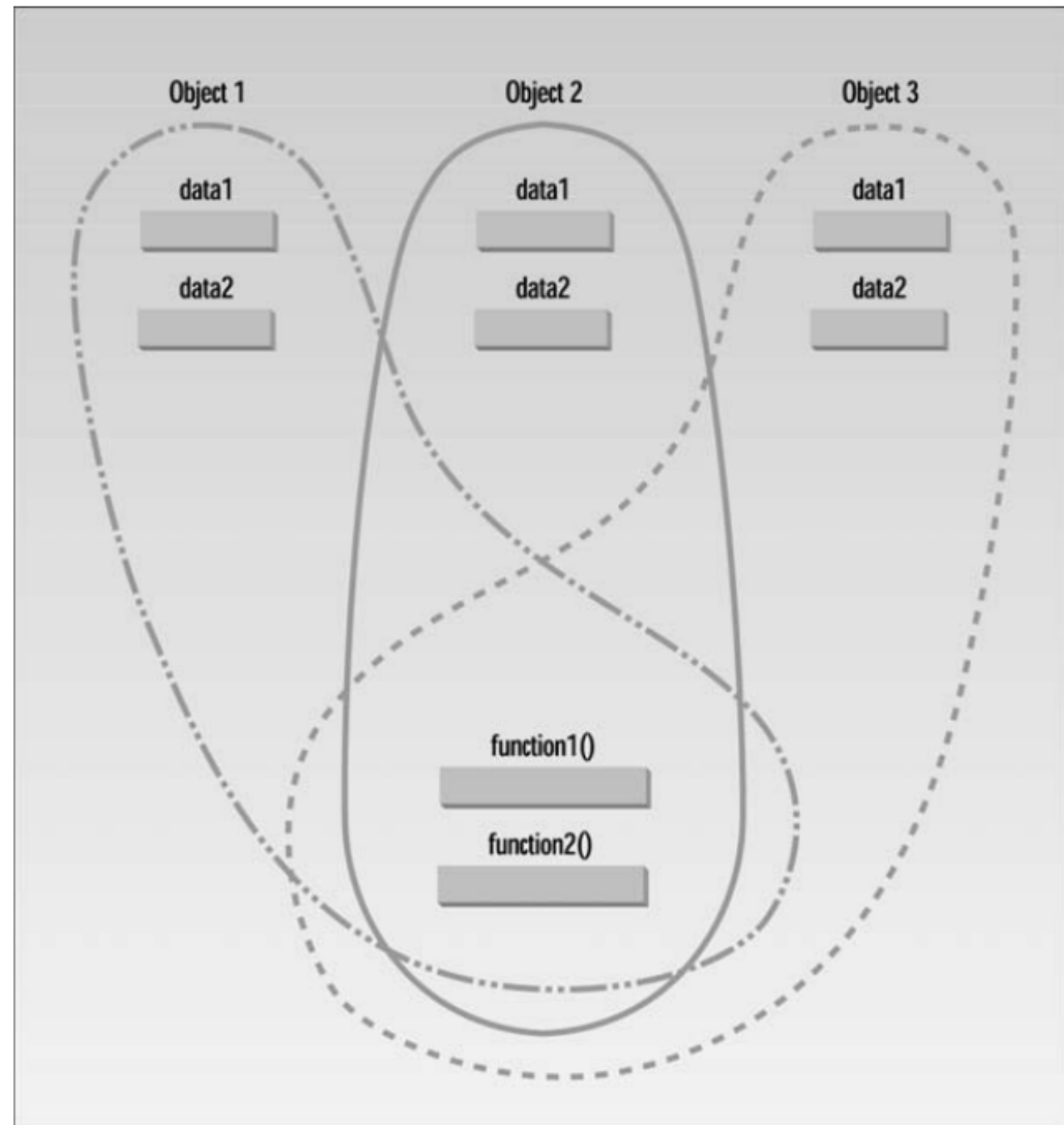
We can overload the operator “=” for deep copy from another object

Rule of 3

A note about Classes, Objects, and Memory

Classes and Object Memory

- In terms of internal memory layout,
- All the objects in a given class use the same member functions.
- The member functions are created and placed in memory only once—when they are defined in the class definition.



The rule of 3

Destructors

Destructors

- In order to deallocate memory allocated to an object, a special member function called a *destructor* must be invoked.
- A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde:

```
class Foo
{
private:
    int data;
public:
    Foo() : data(0)           //constructor (same name as class)
    { }
    ~Foo()                   //destructor (same name with tilde)
    { }
};
```



Destructors – Some Rules

- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object created by the constructor. Hence, destructor cannot be overloaded.
- It cannot be declared static or const.
- Destructor neither requires any argument nor returns any value.
- It is automatically called when an object goes out of scope.
- Destructor release memory space occupied by the objects created by the constructor.
- In destructor, objects are destroyed in the reverse of an object creation.



Summary

- We learned about static class data
- We learned about RAII
- We learned about Rule of 3
 - How the C++ default constructors are not enough sometimes (shallow copy vs deep copy)
 - How to overload = operator for copy assignment operator
 - Destructors and their use